

Learning Report on C-Python Integrated Newton-Raphson Project

1 Introduction

The purpose of this project was to implement the Newton–Raphson method using the C programming language while simultaneously utilizing Python’s symbolic computation abilities. The project demonstrates how two very different programming paradigms—compiled C and interpreted Python—can be combined to create a hybrid numerical system that is both efficient and flexible.

During this project, I explored how Python’s SymPy library can automatically differentiate user-defined functions, eliminating the need to manually calculate derivatives. This not only reduced the chance of human error but also allowed me to focus more on understanding the underlying numerical method rather than on symbolic manipulation. By embedding Python inside a C program, I learned how real-world scientific software often mixes languages to achieve better performance, accuracy, and usability.

2 Project Objective

The main objectives of this project were:

- To understand the Newton-Raphson root-finding algorithm in depth.
- To learn how to link and embed Python within a C program using `Python.h`.
- To explore the use of symbolic computation via the SymPy library.
- To evaluate mathematical expressions dynamically using the TinyExpr library in C.
- To enhance understanding of interoperability between compiled and interpreted languages.

3 Overview of the Code

The core code is responsible for receiving a mathematical function from the user, passing it to the embedded Python interpreter for symbolic differentiation, and then using both the function and its derivative for Newton–Raphson iterations.

This section of the program highlights how C and Python communicate through dynamically constructed strings executed using `PyRun_SimpleString()`. Once the derivative is computed, the C program retrieves its string form and prepares it for numerical evaluation using `TinyExpr`. The code is modular, separating Python operations, string processing, and numerical iteration into different logical steps.

Code Snippet

```
char command[10000];
sprintf(command, "f = sympify('%s')\n", func);
Pystring(command);
Pystring("fprime = diff(f,x)\n");
Pystring("fprime_str = str(fprime)\n");
Pystring("print(\"f'(x)= \", fprime)");
Pystring("import math\n"
"f_func = lambdify(x,f,['math'])\n" //f_func defined in __main__ module
"fprime_func = lambdify(x,fprime,['math'])");
```

4 Working Principle

1. Input and Function Parsing

The program accepts a user-defined mathematical function in Python syntax along with an initial guess.

```
printf("Enter the function (in Python syntax, e.g., x**2 - 3*x + 2): ")
;
fgets(func, sizeof(func), stdin);
func[strcspn(func, "\n")] = 0; // remove newline
```

2. Python Integration

Python is embedded within C using `Py_Initialize()` and `PyRun_SimpleString()` to compute the derivative symbolically.

```
Py_Initialize();
PyRun_SimpleString(
"import sys\n"
"import sympy\n"
);
```

```
printf("\nf(x) = %s", func);
printf("\nf'(x) = %s\n", fprime_str);
printf("\nPython derivative done. Starting C iteration\n");
Py_Finalize();
```

3. C-Side Evaluation

`TinyExpr` compiles and evaluates both $f(x)$ and $f'(x)$ numerically.

```

for (int i = 0; func[i]; i++) {    //loop because tinyexpr takes ** as ^
    if (func[i] == '*' && func[i+1] == '*') {
        func[i] = '^';
        memmove(&func[i+1], &func[i+2], strlen(&func[i+2]) + 1);
    }
}
for (int i = 0; fprime_str[i]; i++) {
    if (fprime_str[i] == '*' && fprime_str[i+1] == '*') {
        fprime_str[i] = '^';
        memmove(&fprime_str[i+1], &fprime_str[i+2], strlen(&fprime_str[
            i+2]) + 1);
    }
}
}

```

4. Iteration Process

The Newton-Raphson update rule used is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Iterations continue until the difference between successive values is below the specified tolerance.

```

printf("\nNewton-Raphson Iterations:\n");
double fx, fpx;
te_expr *expr_f, *expr_fp;
//compiled expressional object representing f(x)
te_variable vars[] = { {"x", &x0} };
// defines the variable x in the expression and points the value of x0
do {
    expr_f = te_compile(func, vars, 1, 0);
    //{te_compile(string func , variable arr , no. of variables,
    pointer to an error position)}
    fx = te_eval(expr_f);
    //evaluates the function with the value of x0
    te_free(expr_f);
    //frees the memory used by expr_f
    expr_fp = te_compile(fprime_str, vars, 1, 0);
    fpx = te_eval(expr_fp);
    te_free(expr_fp);
    if (fpx == 0) {
        printf("Derivative zero at x = %lf\n", x0);
        break;
    }
    x1 = x0 - fx / fpx;
    printf("Iter %2d: x = %.10lf\n", iteration + 1, x1);
    if (fabs(x1 - x0) < tolerance) {
        printf("Converged within tolerance after %d iterations.\n",
            iteration + 1);
        break;
    }
    x0 = x1;
    iteration++;
} while (iteration < 100);
printf("\nApproximate Root = %lf\n", x1);

```

5 Key Learnings and Technical Insights

The development of this hybrid Newton–Raphson solver allowed me to explore how two fundamentally different programming paradigms can cooperate within one system. By combining Python’s symbolic power with C’s numerical speed, I discovered how modern scientific software leverages multiple languages to achieve better performance and accuracy. The project also helped me understand the internal mechanisms behind symbolic differentiation and runtime evaluation.

- Learned to embed Python within C using the `Python.h` API, including interpreter setup, executing dynamic Python code, and finalizing execution safely.
- Understood the complementary nature of symbolic computation (SymPy) and numerical evaluation (TinyExpr in C).
- Gained experience generating symbolic derivatives in Python and evaluating them numerically in C.
- Strengthened understanding of runtime string manipulation, memory management, and error-handling in C.
- Observed how hybrid systems can combine flexibility, speed, and precision better than single-language solutions.

6 Challenges Faced

This project exposed several real-world technical obstacles that occur when integrating multiple languages in a single workflow. Many difficulties arose from differences in syntax, data handling, and compilation requirements between C and Python. Overcoming these challenges helped deepen my debugging skills and improved my ability to design stable and cross-compatible systems.

- Faced difficulty configuring GCC with correct Python include and library paths.
- Required safe conversion of user-provided function syntax from Python’s `**` operator to TinyExpr’s `^` format.
- Had to handle Python-side errors during symbolic differentiation and ensure proper communication between C and Python.
- Encountered runtime issues such as division by zero, lack of convergence, and invalid mathematical expressions.
- Debugged multi-language execution errors arising from syntax mismatches and memory mismanagement.

7 Outcome and Reflections

The final outcome of the project was a successful and reliable C–Python integrated numerical solver capable of performing symbolic differentiation and iterative root-finding. Completing this system enhanced my understanding of numerical algorithms and strengthened

my technical confidence in working with external libraries and cross-language communication. It also highlighted the importance of modular design in computational projects.

- Developed a fully operational hybrid system combining Python’s symbolic engine with C’s numerical evaluator.
- Deepened understanding of Newton–Raphson iteration, convergence properties, and numerical stability.
- Improved skills in modular programming, library integration, and interoperability across languages.
- Learned how hybrid architectures can produce efficient and maintainable scientific computing tools.
- Gained exposure to real-world computational workflows involving multiple programming ecosystems.

8 Conclusion

This project demonstrated that combining high-level symbolic tools with low-level numerical methods can create powerful and flexible computational systems. Integrating C and Python not only improved performance and expressiveness but also provided valuable practical experience in multi-language software engineering. Overall, the project has strengthened my foundation in both numerical methods and systems programming.

- Proved the effectiveness of C–Python interoperability for building accurate and adaptable solvers.
- Demonstrated how symbolic computation enhances numerical algorithms by simplifying derivative handling.
- Strengthened my understanding of hybrid programming approaches in scientific computation.
- Reinforced the relevance of multi-language design in modern engineering and research applications.